`ffmt`: Checked C++ Formatting in Accelerator Kernels

Working fork development notes

22 October 2026 — working revision 3

Abstract

Readable diagnostics inside accelerator kernels require both device-callable formatting and an output transport with known behavior. We describe a C++20 implementation that shares one checked formatting engine across host, CUDA, HIP, SYCL and OpenMP target code. Output iterators provide bounded memory formatting, while direct kernel printing streams through fixed-size staging storage without a required host logging service. An independent, pinned `{fmt}` build supplies exact expected bytes. Qualification separates stdout agreement, device-memory numeric agreement, invalid-format diagnostics, runtime controls, source coverage and performance. We report representative JLSE runs on x86_64, Arm and NVIDIA, AMD and Intel accelerators, including Kokkos routes. We retain failures and limitations rather than counting compilation or skipped execution as conformance. The experiments establish agreement for the sampled corpus and captured software stacks; they do not establish exhaustive fmt or strict freestanding conformance, or performance superiority.

1 Scope and evidence

`ffmt` lets a programmer write brace-style formatting inside a kernel. A successful implementation must validate its syntax and argument types, construct the intended representation, and deliver the resulting bytes. These obligations are tested separately. The implemented library has no mandatory host initialization or drain API; normal runtime synchronization still applies to the kernel and its output.

The original implementation is pinned at `3fe258a87675c997667c899c2b0f0cad405aad5a`. The oracle is fmt commit `0fc31633e7d53ac895ae0a688b83b7d44f0b0c1f`; both repositories and the working fork are identified in the artifact[1, 2]. The supplied 21 October outline is preserved as `outline-v2.tex`. Its two-path architecture and unexecuted-test ledger describe that historical review. This revision describes implemented changes and captured experiments. Tables are generated from selected JSON evidence rather than filled with estimated pass counts.

The supported surface targets device-suitable fmt behavior. It includes runtime and wide formats, positional and named arguments, dynamic specifications, custom formatters, ranges/tuples/join, locale-independent chrono, selected standard type adapters, explicit ANSI styling and typed percent formatting. Filesystem, stream, locale and timezone-database services, RTTI, thread identifiers and exception traversal are outside this device surface. A standard type adapter does not make every implementation of that type device callable.

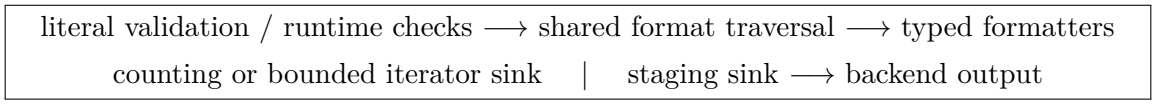
2 Interface and shared engine

```
#include <ffmt/base.hpp>
// Within a supported host function or kernel:
FFMT_PRINTLN("id={0:04d} value={1:.6e} again={0}", id, value);

char buffer[32];
auto result = ffmt::format_to_n(buffer, sizeof buffer,
                                ffmt::runtime(pattern), value);
if (!result) { /* inspect result.error */ }
```

Literal-format constructors validate the syntax and the selected formatter at compilation. Runtime text uses the same parser and returns an error. `format_to_n` reports the number of code units stored and, on success, the complete required size. It never writes beyond the supplied count and does not append a terminator. A short destination is successful bounded formatting. Runtime errors may leave a prefix; the required count is not specified on failure. `try_print` and `try_println` return checked stdout status. The convenience print calls discard it.

The implementation replaces format-to-printf translation with a shared type-directed parser and sink. The same format traversal can count output, write an output iterator or feed the stdout staging sink. Runtime widths and precisions are checked for their type and value. Format text and field counts have no separate fixed descriptor-array limit. The default 128-byte staging size bounds an emission chunk, not the complete output length. Independent calls may interleave between chunks; atomic whole-message printing is not part of the contract.



Formatter contexts expose parsing and output operations without imposing allocation or exception unwinding in the core. Existing static custom formatters returning nested `formatted` values remain accepted. New custom types can derive from another formatter and reuse its parser. Named fields refer to their original values; argument expressions are not repeated to implement repeated positional fields. The API does not create new left-to-right argument-evaluation guarantees.

Padding a family must not accidentally invoke a mutating formatter twice. Bounded pointer destinations render once and move the retained prefix in place; left-aligned streaming also renders once. Right/center alignment on an irreversible sink requires a body proven replayable by the library. Unknown custom formatters and consuming ranges return `unsupported_type` before rendering that body. The implementation does not introduce an arbitrary temporary-output cap to hide this restriction.

3 Conversion and transport

Integer formatting preserves signed magnitude for nondecimal brace formats and checks the type-specific specification. Decimal floating conversion uses integer representations of IEEE values. The ordinary binary32/binary64 shortest path incorporates Dragonbox material adapted through fmt[3, 2]. An exact decimal interval implementation supplies a portable shortest path for SPIR-V. Extended host representations have dedicated conversion rather than narrowing to double. Native extended-format tests distinguish x87 precision from Arm binary128. A separate native `__float128` test uses a transparent oracle formatter adapter because the pinned fmt public type mapper rejects that raw extension type.

Fixed and scientific conversion, hexadecimal conversion, signed zero, subnormals and special encodings are exercised separately. Large requested precision is emitted through the sink, so increasing precision does not require an output-sized temporary. This does not imply that stack use is constant across types or compiler instantiations: numeric scratch state, inlining, spills and device runtime call frames remain relevant.

Narrow memory output preserves bytes, including embedded NUL. Wide memory formatting matches the oracle's code units. Wide stdout is encoded as UTF-8; invalid scalar values and unmatched UTF-16 surrogate halves produce an error. The Unicode tests cover selected width, precision, fill and debug-escaping behavior. They are not a complete Unicode conformance suite.

The host transport uses counted byte output. Native accelerator transports use constant percent-conversion strings to deliver already formatted bytes; formatted payload is not reinterpreted as a percent format. This avoids a dependency on runtime-produced string pointers being accepted by a device printf implementation. Backend output remains a runtime dependency.

The measured Intel SPIR-V runtime drops NUL even through a literal character conversion. The default implementation detects NUL-containing chunks and returns `transport_error`; it does not report successful complete printing. Memory formatting is independent of that limitation. An opt-in macro permits a separately validated binary-safe transport. Runtime output buffer exhaustion, concurrent interleaving and kernel synchronization are distinct from formatting correctness.

AdaptiveCpp's supported SSCP print builtin accepts null-terminated text. The measured route therefore also returns `transport_error` for a chunk containing NUL. An independent builtin probe emits only the prefix before NUL; memory output preserves all bytes. Direct variadic printf failed even for an ASCII control in that stack, so it supplies no alternative binary-safe path. This limitation belongs to the measured SSCP transport, not native HIP printf.

4 Independent conformance method

The oracle source and its implementation object are compiled for the host. Generated target programs include ffmt and backend headers, never fmt. Expected output is captured without trimming it to an ffmt capacity. Each stdout case is invoked in a separate process, and raw byte comparison retains empty output, newlines, invalid UTF-8 and NUL. Payload cannot imitate a harness control record. Harness regressions deliberately test truncation, extra and duplicate bytes, reordering, missing records and unrelated compiler errors.

Every route first builds and runs a positive control and reports the actual selected execution space or device. CUDA/HIP launches check runtime errors; SYCL selects a GPU and waits with exception propagation; OpenMP checks `omp_is_initial_device` with mandatory offload. Kokkos verifies the expected default execution space in stdout and independent memory controls. An unavailable compiler, missing device or rejected control is not a pass.

Compilation failures are recursively isolated from shared source shards. Execution-failing shards are also reduced before assigning per-case outcomes, because a shared switch can change stack allocation or compiler behavior. The failed combined binaries and diagnostics remain in the manifest. This distinguishes individual formatting failures from code-generation or resource failures affecting a larger translation unit.

The numeric memory corpus loads 4,096 seeded binary32 patterns and 4,096 binary64 patterns, plus 24 explicit boundary encodings, from device memory. Each of the 8,216 input records combines default, fixed, scientific, general and hexadecimal formatting. Results, status and required/written

counts are copied back and serialized with checked framing. Thus these counts denote input records, each containing five format checks; they are not 8,216 stdout messages. This suite runs independently even when a stdout control fails.

Invalid-literal testing compares intended rejection in both libraries. Positive and per-file syntax controls prevent missing headers or malformed fixtures from satisfying the negative test. Unrelated diagnostics and compiler crashes have separate statuses. Runtime-invalid text is tested through checked status APIs. Optional compiler types use an explicit unsupported status.

5 Representative JLSE qualification

The selected systems are Rome x86_64, GH200 Arm/CUDA, NVIDIA A40 CUDA, MI300X HIP and Intel Max SYCL/SPIR-V. The H100 queue profile used an explicitly recorded A40 alternative because the primary queue was occupied; this campaign does not establish H100 execution. These are Cobalt allocations, not login-node GPU executions. Profiles retain compiler flags, device identity, driver/runtime discovery, source hashes and commands. Optional toolchains and runtimes are installed under ignored per-system software prefixes with pinned recipes. Kokkos, AdaptiveCpp and chipStar remain distinct routes rather than aliases for native CUDA/HIP/SYCL support.

System	Route	Opt.	Source	Exact/stdout	Exact/numeric
Rome	host-buffer	O0	21f9f5ef	54/54	8216/8216
Rome	host-buffer	O2	21f9f5ef	54/54	8216/8216
Rome	host-specifiers	O0	21f9f5ef	54/54	8216/8216
Rome	host-specifiers	O2	21f9f5ef	54/54	8216/8216
Rome	kokkos-serial	O0	21f9f5ef	54/54	8216/8216
Rome	kokkos-serial	O2	21f9f5ef	54/54	8216/8216
GH200	host-buffer	O0	21f9f5ef	54/54	8216/8216
GH200	host-buffer	O2	21f9f5ef	54/54	8216/8216
GH200	host-specifiers	O0	21f9f5ef	54/54	8216/8216
GH200	host-specifiers	O2	21f9f5ef	54/54	8216/8216
GH200	cuda	O0	21f9f5ef	54/54	8216/8216
GH200	cuda	O2	21f9f5ef	54/54	8216/8216
GH200	nvcc	O0	21f9f5ef	54/54	8216/8216
GH200	nvcc	O2	21f9f5ef	54/54	8216/8216
MI300X	hip	O0	21f9f5ef	54/54	8216/8216
MI300X	hip	O2	21f9f5ef	54/54	8216/8216
MI300X	kokkos-hip	O0	21f9f5ef	54/54	8216/8216
MI300X	kokkos-hip	O2	21f9f5ef	54/54	8216/8216
MI300X	acpp	O0	21f9f5ef	52/54	8216/8216
A40	cuda	O0	21f9f5ef	54/54	8216/8216
A40	cuda	O2	21f9f5ef	54/54	8216/8216
A40	nvcc	O0	21f9f5ef	54/54	8216/8216
A40	nvcc	O2	21f9f5ef	54/54	8216/8216
Intel Max	sycl	O0	21f9f5ef	53/54	8216/8216
Intel Max	sycl	O2	21f9f5ef	53/54	8216/8216
Intel Max	kokkos-sycl	O0	21f9f5ef	53/54	8216/8216
Intel Max	kokkos-sycl	O2	21f9f5ef	53/54	8216/8216
Intel Max	omp-spirv	O0	21f9f5ef	53/54	8216/8216
A40	kokkos-cuda	O0	21f9f5ef	8/8	8216/8216
A40	kokkos-cuda	O2	21f9f5ef	8/8	8216/8216
A40	omp-cuda	O0	21f9f5ef	8/8	execution_failure

System	Route	Opt.	Source	Exact/stdout	Exact/numeric
A40	omp-cuda	O2	21f9f5ef	8/8	8216/8216
A40	acpp	O0	21f9f5ef	7/8	8216/8216
A40	acpp	O2	21f9f5ef	7/8	8216/8216
MI300X	omp-hip	O0	98dba67c	8/8	execution_failure
MI300X	omp-hip	O2	98dba67c	6/8	8216/8216
MI300X	acpp	O0	98dba67c	7/8	8216/8216
MI300X	acpp	O2	98dba67c	7/8	8216/8216
Intel Max	omp-spirv	O2	98dba67c	4/8	8216/8216
Intel Max	chipstar	O0	98dba67c	8/8	8216/8216
Intel Max	chipstar	O2	98dba67c	8/8	8216/8216

Selected captured qualification configurations. Stdout and numeric denominators are separate. A smaller exact count and every non-pass numeric status retain a gap; a table row is not a blanket backend-support claim. Per-case evidence, source snapshots and diagnostics are in the companion JSON.

Strict configurations use `-fno-fast-math` at both optimization levels. Host sanitizers and strict warnings exercise bounded canaries, parser errors, wide data and custom protocols. Fast-math is a separate assessment: a formatter cannot reconstruct information destroyed by earlier user arithmetic. JIT startup limits are recorded. One Intel optimized numeric binary initially timed out at 120 seconds; the identical binary subsequently matched all 8,216 records with a longer startup allowance. That timeout is not evidence of a numeric mismatch.

5.1 Source coverage and coverage gaps

Metric	Covered	Instrumented	Percent
Regions	3244	3519	92.19
Executable lines	3445	3740	92.11
Branch edges	2119	2540	83.43

Source coverage unions executable positions across independently profiled binaries and template instantiations at snapshot `0c2cb76`, including the final counted-view validation repair. It reports regions, executable lines and true/false branch edges for all instrumented ffmt headers. The denominator excludes uninstantiated templates, compile-time evaluation and device-only code. It therefore cannot establish hardware coverage or completeness of the fmt feature surface. The functional corpus also samples rather than exhausts format strings, types, floating encodings and Unicode text.

6 Measurements and resources

Formatting into preallocated memory and direct printing are measured separately. Inputs and outputs remain observable through exact verification and checksums. The formatting benchmark does not charge only one library for allocating a result string. Host fmt is compared on host memory formatting; it is not represented as a GPU implementation. Native printf is compared only for semantically matched integer/fixed-scientific output specifications. The sample does not compare default shortest conversion with unqualified percent-g formatting.

Each measured group has five warm-up batches and thirty measured batches. The host wall timer for accelerators includes launch and synchronization, with data copies and checksums outside the timed interval. Normalizing batch time by calls gives an amortized figure, not individual GPU-call latency. Intel SYCL additionally records device-event batch times and explicit executable-kernel-bundle preparation with persistent SYCL cache disabled. Backend driver caches can still affect preparation; these measurements are not a claim of hardware-cache-cold compilation. Historical first-batch timing after an earlier verification process is labeled first batch, not cold JIT.

System	Route	Engine	Data	Sink	Headers	ns/call
Intel Max	chipstar	ffmt	float {}	memory	334ca65c	265705.39
Intel Max	sycl	ffmt	float .8e	memory	332a7a7f	30529.35
Intel Max	sycl	ffmt	float {}	memory	334ca65c	261119.40
Intel Max	sycl	ffmt	float .8e	print	332a7a7f	69285.14
Intel Max	sycl	ffmt	integer	memory	332a7a7f	11340.19
Intel Max	sycl	ffmt	integer	print	332a7a7f	44311.77
Intel Max	sycl	printf	float .8e	print	332a7a7f	7080.85
Intel Max	sycl	printf	integer	print	332a7a7f	6560.48
Arm/GH200	host-buffer	ffmt	float .8e	memory	332a7a7f	122.77
Arm/GH200	host-buffer	ffmt	float .8e	print	332a7a7f	130.50
Arm/GH200	host-buffer	ffmt	integer	memory	332a7a7f	98.54
Arm/GH200	host-buffer	ffmt	integer	print	332a7a7f	97.88
Arm/GH200	host-buffer	fmt	float .8e	memory	332a7a7f	53.84
Arm/GH200	host-buffer	fmt	integer	memory	332a7a7f	48.94
Arm/GH200	host-buffer	printf	float .8e	print	332a7a7f	135.75
Arm/GH200	host-buffer	printf	integer	print	332a7a7f	54.00
Arm/GH200	host-specifiers	ffmt	float .8e	memory	332a7a7f	133.48
Arm/GH200	host-specifiers	ffmt	float .8e	print	332a7a7f	131.50
Arm/GH200	host-specifiers	ffmt	integer	memory	332a7a7f	95.59
Arm/GH200	host-specifiers	ffmt	integer	print	332a7a7f	101.50
Arm/GH200	host-specifiers	fmt	float .8e	memory	332a7a7f	53.00
Arm/GH200	host-specifiers	fmt	integer	memory	332a7a7f	42.45
Arm/GH200	host-specifiers	printf	float .8e	print	332a7a7f	134.50
Arm/GH200	host-specifiers	printf	integer	print	332a7a7f	55.50
A40	nvcc	ffmt	float .8e	memory	332a7a7f	6726.67
A40	nvcc	ffmt	float .8e	print	332a7a7f	222867.13
A40	nvcc	ffmt	integer	memory	332a7a7f	4096.61
A40	nvcc	ffmt	integer	print	332a7a7f	138106.19
A40	nvcc	printf	float .8e	print	332a7a7f	29305.85
A40	nvcc	printf	integer	print	332a7a7f	27100.23
MI300X	hip	ffmt	float .8e	memory	332a7a7f	17375.98
MI300X	hip	ffmt	float .8e	print	332a7a7f	659792.50
MI300X	hip	ffmt	integer	memory	332a7a7f	10411.69
MI300X	hip	ffmt	integer	print	332a7a7f	499450.78
MI300X	hip	printf	float .8e	print	332a7a7f	50579.27
MI300X	hip	printf	integer	print	332a7a7f	50848.38
Rome	host-buffer	ffmt	float .8e	memory	332a7a7f	166.45
Rome	host-buffer	ffmt	float .8e	print	332a7a7f	212.81
Rome	host-buffer	ffmt	integer	memory	332a7a7f	107.81
Rome	host-buffer	ffmt	integer	print	332a7a7f	143.12
Rome	host-buffer	fmt	float .8e	memory	332a7a7f	93.98
Rome	host-buffer	fmt	integer	memory	332a7a7f	77.42
Rome	host-buffer	printf	float .8e	print	332a7a7f	315.94
Rome	host-buffer	printf	integer	print	332a7a7f	110.62
Rome	host-specifiers	ffmt	float .8e	memory	e73f2c71	188.89
Rome	host-specifiers	ffmt	float .8e	print	e73f2c71	199.61
Rome	host-specifiers	ffmt	integer	memory	e73f2c71	110.25
Rome	host-specifiers	ffmt	integer	print	e73f2c71	130.86
Rome	host-specifiers	fmt	float .8e	memory	e73f2c71	90.43
Rome	host-specifiers	fmt	integer	memory	e73f2c71	73.66
Rome	host-specifiers	printf	float .8e	print	e73f2c71	235.16
Rome	host-specifiers	printf	integer	print	e73f2c71	102.34

Representative measured groups: one call, one translation unit and 128-byte staging. Values are

median batch time divided by calls. Host and device execution models differ; cross-system rows are not speedup claims. The full artifact contains 768 measured groups, including other capacities, two calls and multiple translation units. For overlapping groups, the table selects the latest captured measured run. The header fingerprint identifies its exact source map in `paper-selection.json`; historical measurements are not relabeled as the final revision.

Resource probes separately instantiate integer/floating formats, memory/stdout sinks and one/two calls at staging sizes 16, 64, 128 and 512. Fresh output directories prevent object reuse for compilation timing, although filesystem caches are not evicted. Objects, final executables, tool reports and hashes are retained. Compiler static-frame reports exclude cumulative call depth; device stack frames, spills, register counts, read-only sections and executable size are different quantities. A source staging array is not evidence that all of it occupies shared local memory, nor does an object-size change alone measure runtime occupancy. Unsupported resource-report fields remain absent.

GPU target	Route	Stage	Sink	Calls	Regs/VGPR	Frame/scratch B
gfx942	hip	128	memory	1	97	1440
gfx942	hip	128	memory	2	64	1520
gfx942	hip	128	print	1	124	1600
gfx942	hip	128	print	2	64	1744
gfx942	hip	512	memory	1	97	1440
gfx942	hip	512	memory	2	64	1520
gfx942	hip	512	print	1	124	1984
gfx942	hip	512	print	2	64	2128
sm_86	nvcc	128	memory	1	120	1392
sm_86	nvcc	128	memory	2	120	1400
sm_86	nvcc	128	print	1	70	1568
sm_86	nvcc	128	print	2	103	1664
sm_86	nvcc	512	memory	1	120	1392
sm_86	nvcc	512	memory	2	120	1400
sm_86	nvcc	512	print	1	70	1952
sm_86	nvcc	512	print	2	104	2432

These compile-only shortest-format probes target the named GPU architecture; they do not require that GPU on the compilation node. They report CUDA compiler stack-frame bytes or AMD scratch bytes per lane, respectively; those columns are not equivalent cross-vendor measures. Registers are CUDA registers or AMD VGPRs. Device-image symbol inspection also distinguishes read-only formatting tables before and after linking. The captured CUDA multi-translation-unit build retains two sets of four tables (4,096 bytes), versus one set (2,048 bytes) in a single translation unit. Multiple call sites within one translation unit do not independently duplicate those tables in this probe.

7 Limits and reproducibility

Results belong to the captured source and runtime stacks, not to every release sharing a backend name. Compile-only minimized probes are labeled accordingly. The artifact records failures before and after fixes, including device-callability of tuple construction, nonrecursive chrono lowering, matched OpenMP compiler/runtime requirements and numeric launch resource failures. No upstream compiler issue is claimed filed by this experiment.

The stdout conformance probes use one active work item per case. They do not establish concurrent-message behavior or printf FIFO saturation limits. The printing measurements use bounded

experiment volumes. No separate no-op launch baseline, `std::format` or `FMT_COMPILE` comparison was collected; the reported host-wall batch timing includes launch overhead rather than subtracting an estimated baseline.

The matched ROCm OpenMP route has an optimization-dependent failure. A 97-line independent callback reproducer with no fmt or ffmt dependency aborts at a null GPU address at `-O0`, while the identical source passes 8,216 records at `-O2`. A host OpenMP control passes under address and undefined-behavior sanitizers. This isolates a compiler/runtime boundary for that captured stack; it does not establish that all OpenMP failures have the same cause. Source, exact commands and outcomes accompany the artifact.

The NVIDIA OpenMP numeric probe also reports an illegal access at `-O0`, while its optimized numeric run passes. Focused OpenMP family builds can exceed the recorded compiler timeout; an optimized Intel padding binary additionally fails module creation with an unresolved overflow intrinsic. These outcomes remain gaps rather than exact-output passes. The independent AMD reproducer does not by itself explain the other failures.

The library is header-only and exports a CMake interface target. Make and CTest run the focused and independent suites; the JLSE helper submits allocation-owned jobs. The README and evidence guide provide exact commands. Raw installations, logs and binaries are untracked; compact evidence and pinned recipes are tracked. The fork's stacked PRs separate harness, engine, family, portability, qualification and measurement changes. Further systems can use the same oracle and record schema without converting a skipped or unexecuted configuration into a passing result.

References

[1] T. Applencourt, *ffmt*, original source and working fork. https://github.com/TApplencourt/ffmt, https://github.com/nscottnichols/ffmt. Immutable revisions and per-file hashes are recorded in the evidence manifests.

[2] V. Zverovich and contributors, *fmt*, oracle source. https://github.com/fmtlib/fmt/tree/0fc31633e7d53ac895ae0a688b83b7d44f0b0c1f.

[3] J. Jeon and contributors, *Dragonbox*. https://github.com/jk-jeon/dragonbox. Adaptation and component licenses are identified in the artifact's NOTICE and LICENSES directory.